

SKILL SESSION 21&22

Session 21:

TASK 1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

UW PICO 5.09

File: module.h

```
#ifndef MODULE_H
#define MODULE_H

int add_numbers(int a, int b);
int multiply_numbers(int a, int b);
void display_result(int result);

#endif
```

```
#include <stdio.h>
#include "module.h"

int add_numbers(int a, int b)
{
    return a + b;
}

int multiply_numbers(int a, int b)
{
    return a * b;
}

void display_result(int result)
{
    printf("Result: %d\n", result);
}
```

```
#include <stdio.h>
#include "module.h"

int main()
{
    int a = 10;
    int b = 5;
    int sum;
    int product;

    printf("Module integration test started.\n");

    if (a < 0 || b < 0)
    {
        printf("Error: Invalid input.\n");
        return 1;
    }

    sum = add_numbers(a, b);
    product = multiply_numbers(a, b);

    printf("Addition: ");
    display_result(sum);

    printf("Multiplication: ");
    display_result(product);

    printf("End-to-end execution completed successfully.\n");

    return 0;
}
```

```
[(base) saiabburi@Charishma-MacBook-Air ~ % mkdir Skill11
```

```
[(base) saiabburi@Charishma-MacBook-Air ~ % cd Skill11
```

```
[(base) saiabburi@Charishma-MacBook-Air Skill11 % nano module.h
```

```

[(base) saiabburi@Charishma-MacBook-Air Skill11 % nano module.c
[(base) saiabburi@Charishma-MacBook-Air Skill11 % nano main.c
[(base) saiabburi@Charishma-MacBook-Air Skill11 % gcc -Wall -Wextra main.c module.c -o task1
[(base) saiabburi@Charishma-MacBook-Air Skill11 % ./task1
Module integration test started.
Addition: Result: 15
Multiplication: Result: 50
End-to-end execution completed successfully.
[(base) saiabburi@Charishma-MacBook-Air Skill11 % ls
main.c          module.c        module.h        task1
(base) saiabburi@Charishma-MacBook-Air Skill11 % █

```

TASK 2: To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```

[(base) saiabburi@Charishma-MacBook-Air Skill11 % nano task2.c
[(base) saiabburi@Charishma-MacBook-Air Skill11 % gcc -Wall -Wextra task2.c -o task2
[(base) saiabburi@Charishma-MacBook-Air Skill11 % echo "Test data" > test.txt
[(base) saiabburi@Charishma-MacBook-Air Skill11 % ./task2 test.txt
Runtime error handling test started.
File opened successfully.
Resources released successfully.
Task 2 completed.
[(base) saiabburi@Charishma-MacBook-Air Skill11 % ./task2 missing.txt
Runtime error handling test started.
Error: Unable to open file.
Recovering gracefully...
[(base) saiabburi@Charishma-MacBook-Air Skill11 % cat error.log
File opening failed.
[(base) saiabburi@Charishma-MacBook-Air Skill11 % ./task2
Runtime error handling test started.
Error: Please provide a file name.
[(base) saiabburi@Charishma-MacBook-Air Skill11 % cat error.log
File opening failed.
Missing file name argument.

```

```
#include <stdio.h>
#include <stdlib.h>

void log_error(const char *message)
{
    FILE *log = fopen("error.log", "a");

    if (log != NULL)
    {
        fprintf(log, "%s\n", message);
        fclose(log);
    }
}

int main(int argc, char *argv[])
{
    FILE *file;

    printf("Runtime error handling test started.\n");

    if (argc != 2)
    {
        printf("Error: Please provide a file name.\n");
        log_error("Missing file name argument.");
        return 1;
    }

    file = fopen(argv[1], "r");

    if (file == NULL)
    {
        printf("Error: Unable to open file.\n");
        log_error("File opening failed.");

        printf("Error: Unable to open file.\n");
        log_error("File opening failed.");
        printf("Recovering gracefully...\n");
        return 1;
    }

    printf("File opened successfully.\n");

    fclose(file);

    printf("Resources released successfully.\n");
    printf("Task 2 completed.\n");

    return 0;
}
```

```
[(base) saiabburi@Charishma-MacBook-Air Skill11 % nano syntax_error.c
[(base) saiabburi@Charishma-MacBook-Air Skill11 % gcc syntax_error.c -o syntax_error
syntax_error.c:5:54: error: expected ';' after expression
    printf("This program contains a syntax error.\n")
                                                    ^
                                                    ;
1 error generated.
```

UW PICO 5.09

File: syntax_error.c

```
#include <stdio.h>

int main()
{
    printf("This program contains a syntax error.\n")
    return 0;
}
```

Session 22 :

TASK 3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
((base) saiabburi@Charishma-MacBook-Air Skill11 % nano memory_leak.c
((base) saiabburi@Charishma-MacBook-Air Skill11 % gcc -g -Wall -Wextra memory_leak.c -o memory_leak
((base) saiabburi@Charishma-MacBook-Air Skill11 % ./memory_leak
Memory allocated successfully.
Memory leak test completed.
```

UW PICO 5.09

File: memory_leak.c

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *numbers;

    numbers = malloc(5 * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        numbers[i] = i + 1;
    }

    printf("Memory allocated successfully.\n");
    printf("Memory leak test completed.\n");

    return 0;
}
```

```
(base) saiabburi@Charishma-MacBook-Air Skill11 % leaks --atExit -- ./memory_leak
memory_leak(2171) MallocStackLogging: could not tag MSL-related memory as no_footprint, so those pages will be included in process footprint - (null)
memory_leak(2171) MallocStackLogging: recording malloc and VM allocation stacks using lite mode
Memory allocated successfully.
Memory leak test completed.
Process 2171 is not debuggable. Due to security restrictions, leaks can only show or save contents of readonly memory of restricted processes.

Process:          memory_leak [2171]
Path:             /Users/USER/*/memory_leak
Load Address:     0x102b20000
Identifier:       memory_leak
Version:          0
Code Type:       ARM64
Platform:        macOS
Parent Process:   leaks [2170]

Date/Time:        2026-09-28 09:25:15.991 +0530
Launch Time:      2026-09-28 09:25:15.795 +0530
OS Version:       macOS 13.4.1 (22F82)
Report Version:   7
Analysis Tool:    /usr/bin/leaks

Physical footprint:      2929K
Physical footprint (peak): 2929K
Idle exit:               untracked
-----

leaks Report Version: 4.0, multi-line stacks
Process 2171: 223 nodes malloced for 17 KB
Process 2171: 1 leak for 32 total leaked bytes.

STACK OF 1 INSTANCE OF 'ROOT LEAK: <malloc in main>':
2  dyld                                0x19d693f28 start + 2236
1  memory_leak                        0x102b23ea4 main + 24   memory_leak.c:8
0  libsystem_malloc.dylib             0x19d82cd88 _malloc_zone_malloc_instrumented_or_legacy + 128
=====
```

```
(base) saiabburi@Charishma-MacBook-Air Skill11 % nano memory_leak.c
(base) saiabburi@Charishma-MacBook-Air Skill11 % gcc -g -Wall -Wextra memory_leak.c -o memory_leak
(base) saiabburi@Charishma-MacBook-Air Skill11 % ./memory_leak
Memory allocated successfully.
Memory leak test completed.
Memory released successfully./n%
```

```
(base) saiabburi@Charishma-MacBook-Air Skill11 % leaks --atExit -- ./memory_leak
```

```
leaks Report Version: 4.0, multi-line stacks
Process 2271: 222 nodes malloced for 17 KB
Process 2271: 0 leaks for 0 total leaked bytes.
```

TASK 4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```
(base) saiabburi@Charishma-MacBook-Air Skill11 % nano calculator.c
(base) saiabburi@Charishma-MacBook-Air Skill11 % gcc -Wall -Wextra calculator.c -o calculator
(base) saiabburi@Charishma-MacBook-Air Skill11 % ./calculator
Test 1: 15
Test 2: 50
Test 3: 0
(base) saiabburi@Charishma-MacBook-Air Skill11 % nano test_script.sh
(base) saiabburi@Charishma-MacBook-Air Skill11 % chmod +x test_script.sh
(base) saiabburi@Charishma-MacBook-Air Skill11 % ./test_script.sh
===== AUTOMATED TEST REPORT =====
Compilation: PASS
Test 1: 15
Test 2: 50
Test 3: 0
Test 1: PASS
Test 2: PASS
Test 3: PASS
===== TESTING COMPLETED =====
(base) saiabburi@Charishma-MacBook-Air Skill11 % ./test_script.sh > test_report.txt
(base) saiabburi@Charishma-MacBook-Air Skill11 % cat test_report.txt
===== AUTOMATED TEST REPORT =====
Compilation: PASS
Test 1: 15
Test 2: 50
Test 3: 0
Test 1: PASS
Test 2: PASS
Test 3: PASS
===== TESTING COMPLETED =====
```

UW PICO 5.09

File: calculator.c

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int main()
{
    printf("Test 1: %d\n", add(10, 5));
    printf("Test 2: %d\n", add(20, 30));
    printf("Test 3: %d\n", add(-5, 5));

    return 0;
}
```

```
#!/bin/bash
echo "==== AUTOMATED TEST REPORT ====="

gcc -Wall -Wextra calculator.c -o calculator

if [ $? -ne 0 ]; then
    echo "Compilation: FAIL"
    exit 1
fi

echo "Compilation: PASS"

output=$(./calculator)

echo "$output"

if echo "$output" | grep -q "Test 1: 15"; then
    echo "Test 1: PASS"
else
    echo "Test 1: FAIL"
fi

if echo "$output" | grep -q "Test 2: 50"; then
    echo "Test 2: PASS"
else
    echo "Test 2: FAIL"
fi

if echo "$output" | grep -q "Test 3: 0"; then
    echo "Test 3: PASS"
else
    echo "Test 3: FAIL"
fi

echo "==== TESTING COMPLETED ====="
```